IES Francisco de Goya

Prueba de Conceptos

QuestHub: Comparador de precios de videojuego

- **Video-Test:** https://youtu.be/xPBSqUwhvEA

- **Repositorio:** https://github.com/A1ebyte/ProyectoFCT

- **Curso:** 2º DAW (2025-2026)

- **Integrantes:**
  Kerin Aguilar
  Mohamed Bada
  Freddy De Andrade

1. BackEnd

En el backend hemos desarrollado una API utilizando Spring Boot siguiendo una arquitectura en capas típica de aplicaciones backend.

La estructura del proyecto está organizada en los siguientes paquetes:

- config
- controller
- repository
- service

Este patrón permite separar responsabilidades y mantener el código más organizado y escalable.

Services

```java
package backend.Recetas.service;

import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service  new *
public class PokemonService {   You, A minute ago • Uncommitted changes

    private final RestTemplate restTemplate = new RestTemplate();  1 usage

    public Object getPokemon(String name) {  1 usage  new *
        String url = "https://pokeapi.co/api/v2/pokemon/" + name.toLowerCase();
        return restTemplate.getForObject(url, Object.class);
    }
}
```

En la carpeta service creamos la clase PokemonService.

Su función es encargarse de la lógica de negocio y realizar las peticiones a una API externa. En nuestro caso utilizamos la PokéAPI, que proporciona información sobre los Pokémon.

Como la funcionalidad del proyecto es sencilla, solo implementamos un método que permite obtener un Pokémon por su nombre.

Para realizar la petición HTTP utilizamos RestTemplate, que es una herramienta de Spring que permite consumir APIs externas desde el backend.

El flujo sería:

El frontend solicita un Pokémon al backend.

El backend recibe la petición.

PokemonService realiza una petición HTTP a la PokéAPI.

El backend recibe la respuesta.

Devuelve los datos al frontend.

Controller

```java
package backend.Recetas.controller;

import backend.Recetas.service.PokemonService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController  new *
@RequestMapping("/api/pokemon")
public class PokemonController {

    @Autowired
    private PokemonService pokemonService;

    @GetMapping("/{name}")  new *
    public ResponseEntity<?> obtenerPorNombre(@PathVariable String name) {
            return ResponseEntity.ok(pokemonService.getPokemon(name));
    }
}
You, 11 minutes ago • Uncommitted changes
```

También creamos un controller específico llamado **PokemonController**.

Este controlador expone un **endpoint REST** que permite acceder a la información del Pokémon desde el frontend.

Ejemplo de **endpoint**:

/api/pokemon/{name}

Cuando el frontend hace una petición a este endpoint:

El controlador recibe la solicitud.

Llama al método correspondiente del PokemonService.

El servicio **consulta** la PokéAPI.

Se **devuelve la información** del Pokémon al frontend en **formato JSON**.

Configuración CORS

```java
package backend.Recetas.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration  new *
public class WebConfig {   You, 28 minutes ago • Uncommitted changes

    @Bean  new *
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {  new *
            @Override  no usages  new *
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping( pathPattern: "/**")
                        .allowedOrigins("http://localhost:3000", "http://localhost:5173")
                        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                        .allowedHeaders("*")
                        .allowCredentials(true);
            }
        };
    }
}
```

Además configuramos **CORS** (Cross-Origin Resource Sharing).

Esto **es necesario** porque:

El frontend se ejecuta en un puerto diferente (por ejemplo localhost:3000)

El backend se ejecuta en **localhost:8080**

Sin **CORS**, el navegador **bloquearía** la comunicación entre ambos por motivos de seguridad.

Con esta configuración permitimos que el frontend pueda consumir los endpoints del backend sin problemas.

2. FrontEnd

El frontend está desarrollado con **React**.

El objetivo del componente es:

- Realizar una petición al backend.
- Obtener la información de un Pokémon.
- Mostrar su nombre y su imagen en la interfaz.

```
import { useState } from "react";

function Pokemon() {
  const [pokemon, setPokemon] = useState(null);    "pokemon": Unknown word.

  const fetchPokemon = async () => {
    const res = await fetch("http://localhost:8080/api/pokemon/pikachu");
    const data = await res.json();
    setPokemon(data);
  };

  return (
    <div>
      <button onClick={fetchPokemon}>Cargar Pikachu</button>    "Pikachu": Unknown

      {pokemon && (    "pokemon": Unknown word.
        <div>
          <h2>{pokemon.name}</h2>    "pokemon": Unknown word.
          <img src={pokemon.sprites.front_default} alt={pokemon.name} />    "pokem
        </div>
      )}
    </div>
  );
}

export default Pokemon;
```

- **Función para obtener el Pokémon**

const fetchPokemon = async () => {

Se define una función **asíncrona** porque necesitamos realizar una petición HTTP al backend.

- **Petición al backend**

const res = await fetch("http://localhost:8080/api/pokemon/pikachu");

Aquí utilizamos la función fetch del navegador para realizar una petición **GET** al backend.

La URL indica que estamos llamando al endpoint:

/api/pokemon/pikachu

Este endpoint pertenece al backend desarrollado con Spring Boot.

- **Conversión de la respuesta**

const data = await res.json();

La respuesta del servidor se convierte a **JSON**, que es el formato estándar para intercambiar datos entre frontend y backend.

- **Guardar los datos en el estado**

setPokemon(data);

El resultado se guarda en el estado del componente utilizando el hook useState.

Cuando el estado cambia, React vuelve a renderizar el componente y actualiza la interfaz automáticamente.

- **Renderizado de la información**

Cuando el usuario pulsa el botón:

Cargar Pikachu

se ejecuta **fetchPokemon().**

Una vez recibidos los datos:

- se muestra el **nombre del Pokémon**
- se muestra **su imagen**

Esto se hace mediante renderizado condicional para evitar errores cuando todavía no hay datos cargados.